Credit Card Approval Prediction

Project Description:

Banks and financial institutions receive thousands of credit card applications every day. A significant portion of these are rejected due to factors such as high existing loan balances, insufficient income levels, or excessive credit inquiries. Manually reviewing each application is both time-consuming and highly prone to human error, making it an inefficient process at scale.

This project automates the credit card approval decision using machine learning. By training a predictive model on historical applicant data, the system evaluates financial and demographic inputs to determine whether an applicant is likely to be approved or rejected just as real banks do. Four classification algorithms are applied: Logistic Regression, Random Forest, XGBoost (Gradient Boosting), and Decision Tree.

The best-performing model is saved and integrated into a Flask web application. The project also includes an IBM Watson Machine Learning deployment pipeline, enabling the solution to be hosted on the cloud for scalable, real-time credit card approval predictions accessible through an intuitive user interface.

Scenario 1: Automated Credit Card Application Screening

A bank credit analyst enters a new applicant's financial profile including income type, employment duration, and credit history into the web application. The model returns an instant approval or rejection prediction, enabling the analyst to prioritize applications that require further review.

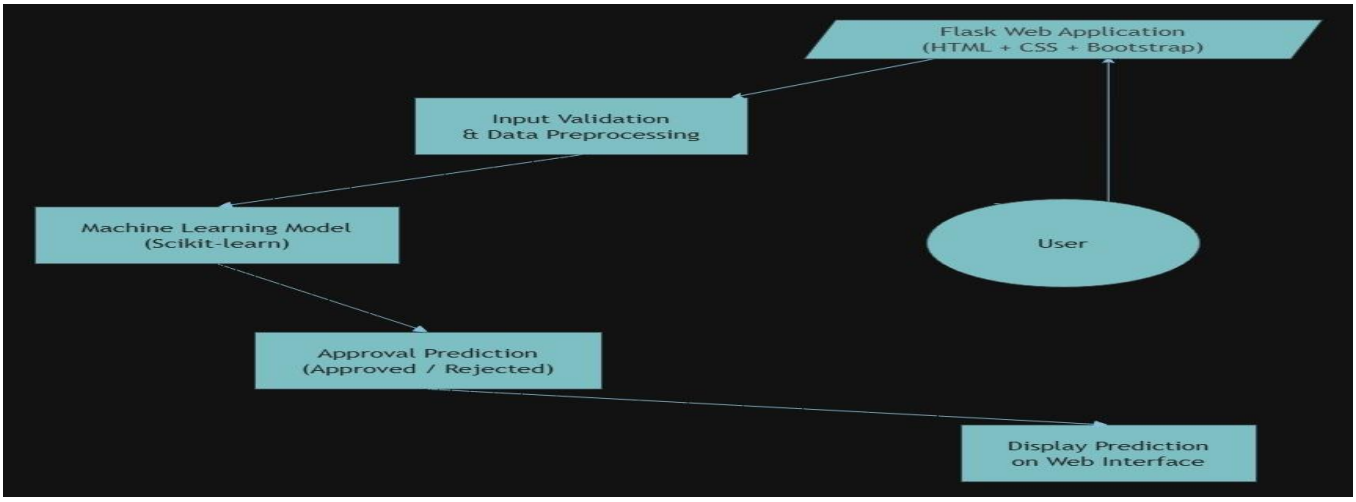
Scenario 2: High-Risk Applicant Identification and Compliance Review

A financial compliance officer uses the platform to batch-screen applicants with past-due loan records. The feature engineering pipeline converts multi-class payment status codes into binary labels, allowing the model to clearly classify high-risk applicants as ineligible for a new credit card.

Scenario 4: Customer Self-Service Credit Card Eligibility Check

A prospective customer uses the web application to enter personal and financial details such as income level, employment status, and credit history before submitting a formal credit card application. The system instantly predicts the likelihood of approval, helping the customer understand their eligibility and reducing unnecessary application rejections.

Technical Architecture:



Pandas:

Description:

The project was developed using a set of powerful Python-based tools and libraries for data processing, machine learning, visualization, and deployment. These technologies provide the foundation for efficient model development, analysis, and application deployment.

(Note: If you are using database in your project definitely you need to add ER Diagram along with description)

Pre-requisites:

- **Flask Framework Knowledge:** Flask Documentation
- **Kaggle Dataset download: https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud**
- **HTML, CSS, and JavaScript Skills:** W3Schools HTML/CSS/JavaScript Tutorials
- **Python Programming Proficiency:** Python Documentation
- **NumPy:** https://numpy.org/doc/stable/
- **Pandas: https://pandas.pydata.org/docs/**
- **Scikit-learn:https://scikit-learn.org/stable/**
- **Matplotlib: https://matplotlib.org/stable/**
- **Seaborn: https://seaborn.pydata.org/**
- **Flask:** https://flask.palletsprojects.com/
- **Version Control with Git:** Git Documentation
- **Development Environment Setup:** Flask Installation Guide

Project Work Flow :

Credit Card Approval Prediction System, you can use the following.

Project Workflow

Milestone 1: Project Planning and Environment Setup

- **Activity 1.1:** Define the project objectives, scope, and requirements for the Credit Card Approval Prediction System.
- **Activity 1.2:** Collect the credit card approval dataset and understand its features.
- **Activity 1.3:** Set up the Python development environment and install required libraries (NumPy, Pandas, Scikit-learn, Flask, Matplotlib, Seaborn).
- **Activity 1.4:** Design the overall system architecture, including frontend, backend, machine learning model, and deployment.

Milestone 2: Data Preprocessing and Model Development

- **Activity 2.1:** Perform data cleaning, handle missing values, encode categorical features, and scale numerical features.
- **Activity 2.2:** Conduct Exploratory Data Analysis (EDA) to identify trends and relationships in the dataset.
- **Activity 2.3:** Train multiple machine learning models and compare their performance.
- **Activity 2.4:** Select the best-performing model and save it for deployment.

Milestone 3: Flask Application Development

- **Activity 3.1:** Develop the Flask backend to load the trained model and process user inputs.
- **Activity 3.2:** Create prediction routes and integrate the machine learning model with the application.
- **Activity 3.3:** Test the prediction functionality and validate system performance.

Milestone 4: Frontend Development

- **Activity 4.1:** Design a responsive user interface using HTML, CSS, Bootstrap, and JavaScript.
- **Activity 4.2:** Develop input forms for applicant details and display prediction results.
- **Activity 4.3:** Integrate the frontend with the Flask backend for real-time predictions.

Milestone 5: Testing, Deployment, and Documentation

- **Activity 5.1:** Perform unit testing, integration testing, and system testing.
- **Activity 5.2:** Deploy the application to IBM Cloud.
- **Activity 5.3:** Prepare the project documentation, architecture diagrams, screenshots, and final report.
- **Activity 5.4:** Upload the project to GitHub and verify all deliverables.

Milestone 6: Conclusion

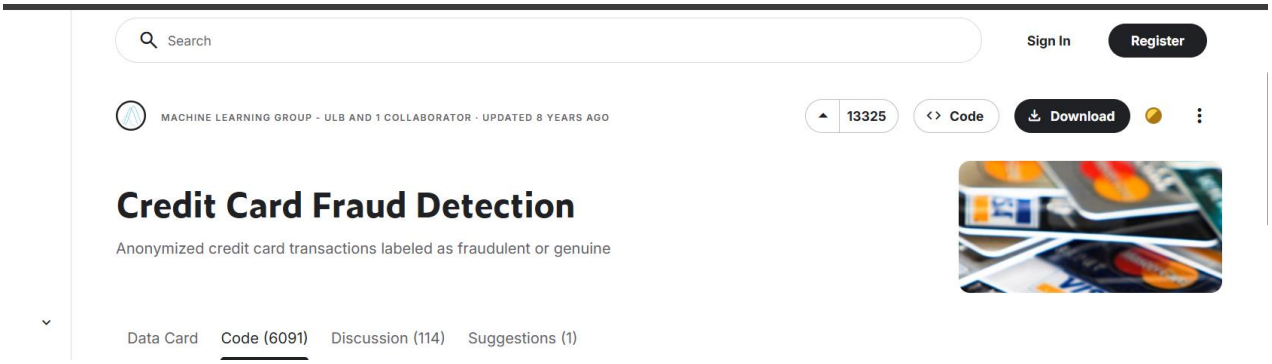
Milestone 1: Project Planning and System Architecture

This milestone focuses on understanding the problem, collecting and analyzing the dataset, selecting the most suitable machine learning algorithm, and designing the overall system architecture. The objective is to establish a strong foundation before beginning model development and deployment.

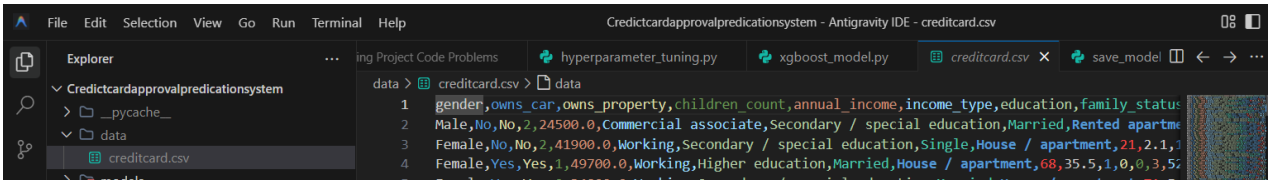
Activity 1.1: Dataset Collection and Preparation

Before building the prediction model, obtain a suitable credit card approval dataset and prepare it for machine learning.

- **Step 1 – Download the Dataset:** Download the Credit Card Approval dataset from Kaggle or the UCI Machine Learning Repository.



Step 2 – Load the Dataset: Load the dataset into a Antigravity using the Pandas library.



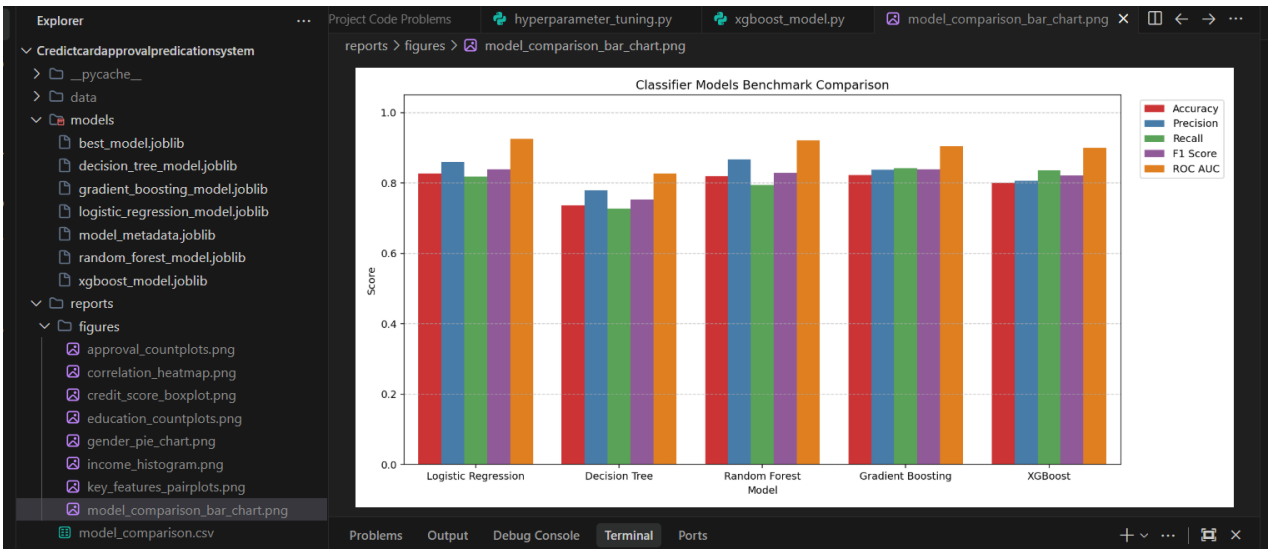
Step 3 – Explore the Dataset: Display the first few records, examine data types, identify missing values, and understand the target variable.

Step 4 – Save the Dataset: Store the dataset in the project's **data/** folder for further preprocessing.

Activity 1.2: Research and Select the Machine Learning Model

Evaluate different machine learning algorithms and choose the model with the highest prediction accuracy.

- Study Logistic Regression.
- Study Decision Tree Classifier.
- Study Random Forest Classifier.
- Compare Accuracy, Precision, Recall, and F1-Score.
- Select the best-performing model for deployment
-



Activity 1.3: Define the System Architecture

Design the architecture of the Credit Card Approval Prediction System.

- Design the system architecture diagram.
- Define Frontend (HTML, CSS, Bootstrap).
- Define Backend (Flask).
- Define Machine Learning module (Scikit-learn).
- Explain data flow from user input to prediction result.

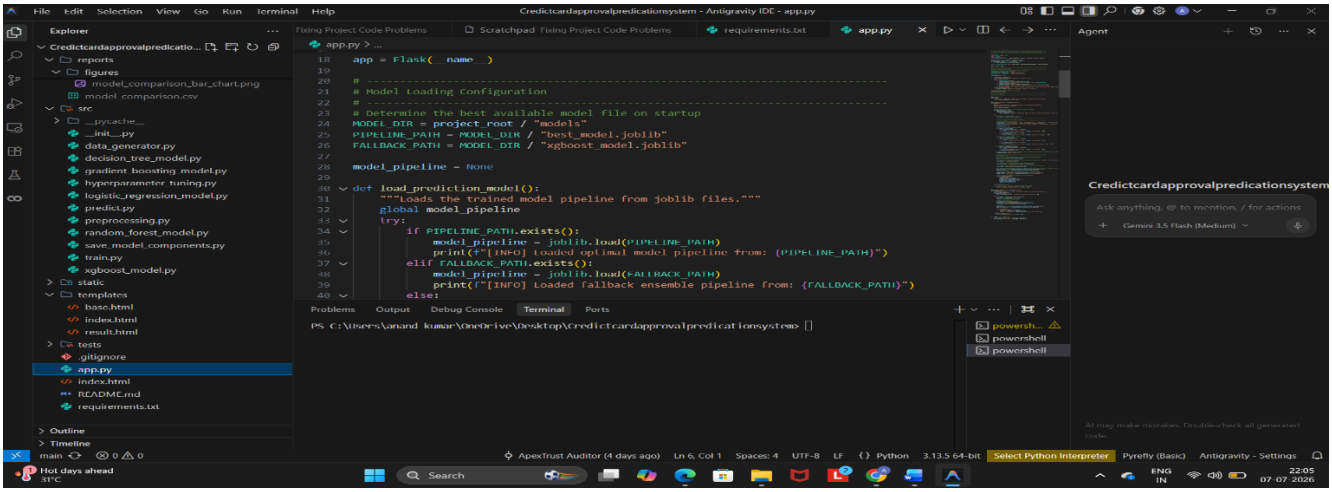
Activity 1.4: Set Up the Development Environment

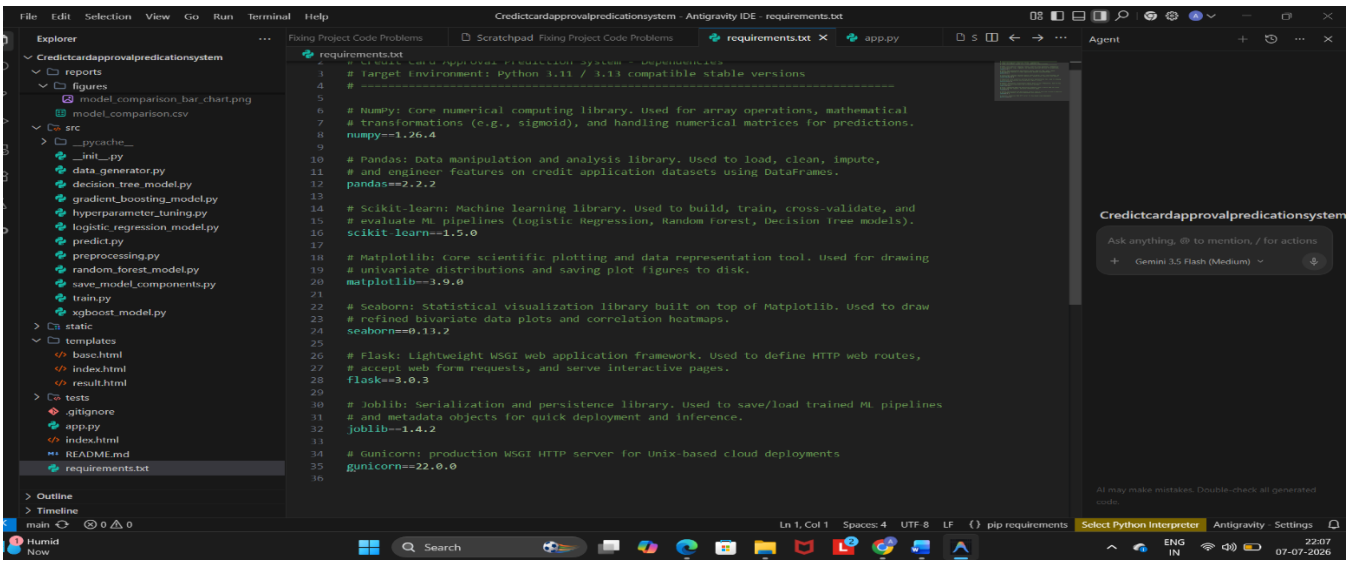
Configure the project development environment.

- Install Python 3.10 or above.
- Create a virtual environment.
- Install required libraries:

pip install pandas numpy matplotlib seaborn scikit-learn flask joblib

- Create the project folder structure.
- Configure Antigravity or Jupyter Notebook.

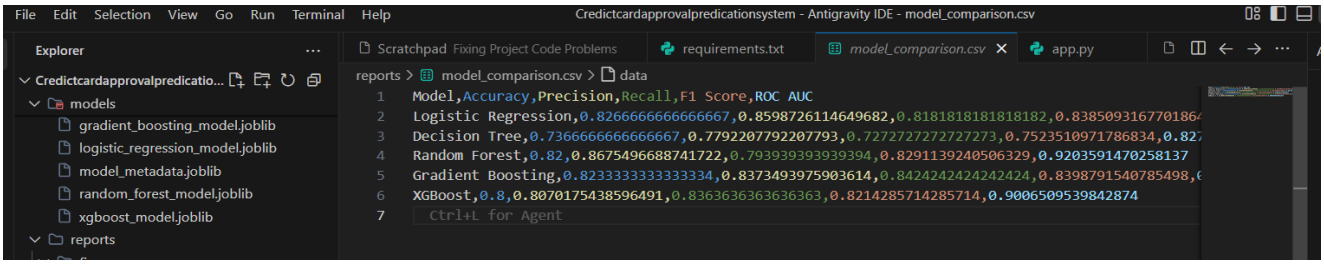

File Edit Selection View Go Run Terminal Help
Credictcardapprovalpredicationsystem - Antigravity IDE - app.py
Explorer
app.py
app = Flask(name)
# Model Loading Configuration
# Determine the best available model file on startup
MODEL_DIR = project_root / "models"
PIPELINE_PATH = MODEL_DIR / "best_model.joblib"
FALLBACK_PATH = MODEL_DIR / "xgboost_model.joblib"
model_pipeline = None
def load_prediction_model():
"""Loads the trained model pipeline from joblib files."""
global model_pipeline
try:
if PIPELINE_PATH.exists():
model_pipeline = joblib.load(PIPELINE_PATH)
print(f"[INFO] Loaded optimal model pipeline from: {PIPELINE_PATH}")
elif FALLBACK_PATH.exists():
model_pipeline = joblib.load(FALLBACK_PATH)
print(f"[INFO] Loaded fallback ensemble pipeline from: {FALLBACK_PATH}")
else:
Problems Output Debug Console Terminal Ports
PS C:\Users\anand kumar\OneDrive\Desktop\Credictcardapprovalpredicationsystem>
Agent
Credictcardapprovalpredicationsystem
Ask anything, @ to mention, / for actions
Gemini 3.5 Flash (Medium)
Select Python Interpreter
Antigravity - Settings


File Edit Selection View Go Run Terminal Help
Credictcardapprovalpredicationsystem - Antigravity IDE - requirements.txt
Explorer
requirements.txt
# Target Environment: Python 3.11 / 3.13 compatible stable versions
# NumPy: Core numerical computing library. Used for array operations, mathematical
# transformations (e.g., sigmoid), and handling numerical matrices for predictions.
numpy==1.26.4
# Pandas: Data manipulation and analysis library. Used to load, clean, impute,
# and engineer features on credit application datasets using DataFrames.
pandas==2.2.2
# Scikit-learn: Machine learning library. Used to build, train, cross-validate, and
# evaluate ML pipelines (Logistic Regression, Random Forest, Decision Tree models).
scikit-learn==1.5.0
# Matplotlib: Core scientific plotting and data representation tool. Used for drawing
# univariate distributions and saving plot figures to disk.
matplotlib==3.9.0
# Seaborn: Statistical visualization library built on top of Matplotlib. Used to draw
# refined bivariate data plots and correlation heatmaps.
seaborn==0.13.2
# Flask: Lightweight WSGI web application framework. Used to define HTTP web routes,
# accept web form requests, and serve interactive pages.
flask==3.0.3
# Joblib: Serialization and persistence library. Used to save/load trained ML pipelines
# and metadata objects for quick deployment and inference.
joblib==1.4.2
# Gunicorn: production WSGI HTTP server for Unix-based cloud deployments
gunicorn==22.0.0
Agent
Credictcardapprovalpredicationsystem
Ask anything, @ to mention, / for actions
Gemini 3.5 Flash (Medium)
Select Python Interpreter
Antigravity - Settings

Milestone 2: Core Functionalities Development

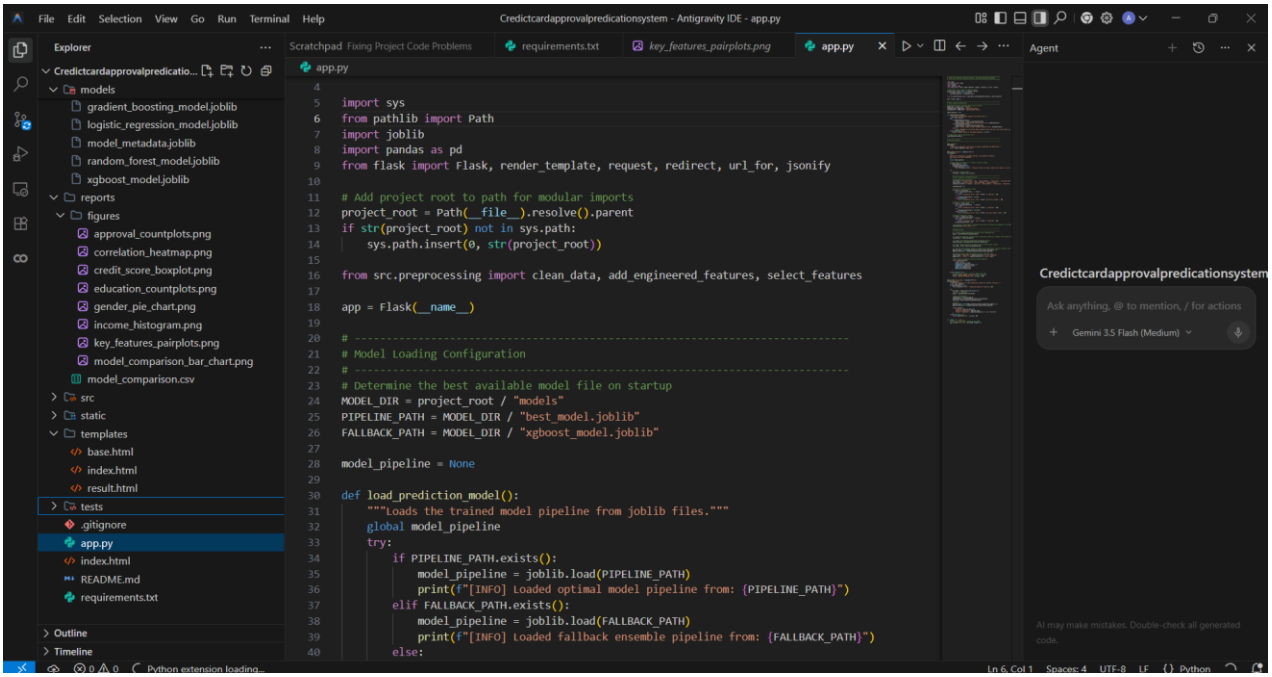
Activity 2.1: Data Preprocessing and Model Development

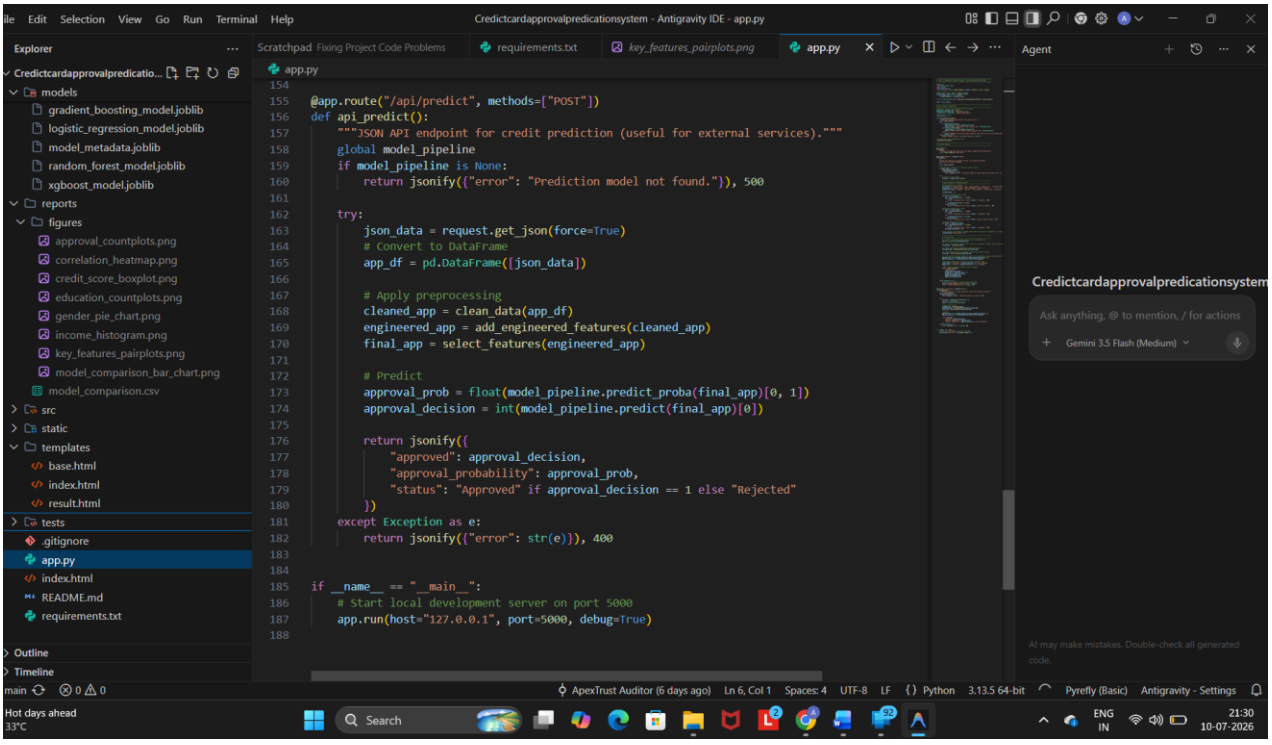
- Handle missing values.
- Encode categorical variables.
- Scale numerical features.
- Split dataset into training and testing sets.
- Train multiple machine learning models.
- Evaluate model performance.
- Save the best model using joblib.



Activity 2.2: Develop the Flask Backend

- Create app.py.
- Load the trained model.
- Accept user inputs.
- Predict credit card approval.
- `Return the prediction to the frontend.


File Edit Selection View Go Run Terminal Help
Credictcardapprovalpredicationsystem - Antigravity IDE - app.py
Explorer
Credictcardapprovalpredicatio...
models
gradient_boosting_model.joblib
logistic_regression_model.joblib
model_metadata.joblib
random_forest_model.joblib
xgboost_model.joblib
reports
figures
approval_countplots.png
correlation_heatmap.png
credit_score_boxplot.png
education_countplots.png
gender_pie_chart.png
income_histogram.png
key_features_pairplots.png
model_comparison_bar_chart.png
model_comparison.csv
src
static
templates
base.html
index.html
result.html
tests
.gitignore
app.py
index.html
README.md
requirements.txt
Outline
Timeline
Scratchpad Fixing Project Code Problems
requirements.txt
key_features_pairplots.png
app.py
app.py
import sys
from pathlib import Path
import joblib
import pandas as pd
from flask import Flask, render_template, request, redirect, url_for, jsonify
# Add project root to path for modular imports
project_root = Path(__file__).resolve().parent
if str(project_root) not in sys.path:
sys.path.insert(0, str(project_root))
from src.preprocessing import clean_data, add_engineered_features, select_features
app = Flask(__name__)
# ---
# Model Loading Configuration
# ---
# Determine the best available model file on startup
MODEL_DIR = project_root / "models"
PIPELINE_PATH = MODEL_DIR / "best_model.joblib"
FALLBACK_PATH = MODEL_DIR / "xgboost_model.joblib"
model_pipeline = None
def load_prediction_model():
"""Loads the trained model pipeline from joblib files."""
global model_pipeline
try:
if PIPELINE_PATH.exists():
model_pipeline = joblib.load(PIPELINE_PATH)
print(f"[INFO] Loaded optimal model pipeline from: {PIPELINE_PATH}")
elif FALLBACK_PATH.exists():
model_pipeline = joblib.load(FALLBACK_PATH)
print(f"[INFO] Loaded fallback ensemble pipeline from: {FALLBACK_PATH}")
else:
Agent
Credictcardapprovalpredicationsystem
Ask anything, @ to mention, / for actions
Gemini 3.5 Flash (Medium)
AI may make mistakes. Double-check all generated code.
Python extension loading...
Ln 6, Col 1
Spaces: 4
UTF-8
LF
Python


Edit Selection View Go Run Terminal Help
Credictcardapprovalpredicationsystem - Antigravity IDE - app.py
Explorer
Credictcardapprovalpredicatio...
models
gradient_boosting_model.joblib
logistic_regression_model.joblib
model_metadata.joblib
random_forest_model.joblib
xgboost_model.joblib
reports
figures
approval_countplots.png
correlation_heatmap.png
credit_score_boxplot.png
education_countplots.png
gender_pie_chart.png
income_histogram.png
key_features_pairplots.png
model_comparison_bar_chart.png
model_comparison.csv
src
static
templates
base.html
index.html
result.html
tests
.gitignore
app.py
index.html
README.md
requirements.txt
Outline
Timeline
Scratchpad Fixing Project Code Problems
requirements.txt
key_features_pairplots.png
app.py
app.py
@app.route("/api/predict", methods=["POST"])
def api_predict():
"""JSON API endpoint for credit prediction (useful for external services)."""
global model_pipeline
if model_pipeline is None:
return jsonify({"error": "Prediction model not found."}), 500
try:
json_data = request.get_json(force=True)
# Convert to DataFrame
app_df = pd.DataFrame([json_data])
# Apply preprocessing
cleaned_app = clean_data(app_df)
engineered_app = add_engineered_features(cleaned_app)
final_app = select_features(engineered_app)
# Predict
approval_prob = float(model_pipeline.predict_proba(final_app)[0, 1])
approval_decision = int(model_pipeline.predict(final_app)[0])
return jsonify({
"approved": approval_decision,
"approval_probability": approval_prob,
"status": "Approved" if approval_decision == 1 else "Rejected"
})
except Exception as e:
return jsonify({"error": str(e)}), 400
if __name__ == "__main__":
# Start local development server on port 5000
app.run(host="127.0.0.1", port=5000, debug=True)
Agent
Credictcardapprovalpredicationsystem
Ask anything, @ to mention, / for actions
Gemini 3.5 Flash (Medium)
AI may make mistakes. Double-check all generated code.
main
ApexTrust Auditor (6 days ago)
Ln 6, Col 1
Spaces: 4
UTF-8
LF
Python
3.13.5 64-bit
Pyrefly (Basic)
Antigravity - Settings
Hot days ahead
33°C
Search
ENG
IN
21:30
10-07-2026

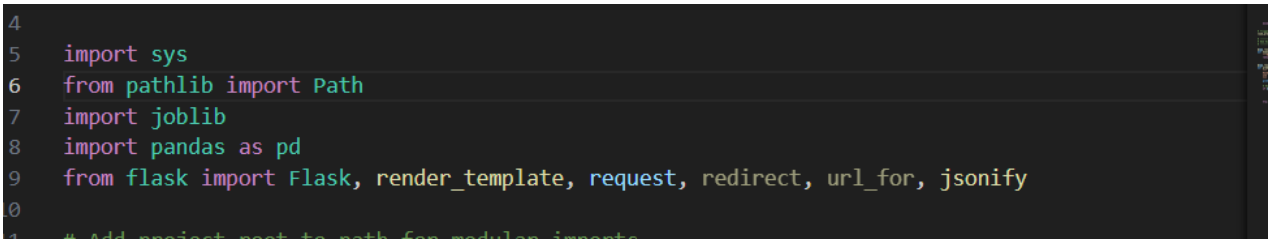
Milestone 3: Flask Application Development

Milestone 3 focuses on developing the Flask backend that connects the trained machine learning model with the user interface. This milestone handles user requests, processes applicant information, performs predictions using the trained model, and returns the approval result.

Activity 3.1: Develop the Main Application (app.py)

Create the Flask Application

- Import Flask, Joblib, NumPy, and required libraries.
- Load the trained machine learning model (model.pkl).
- Create the Flask application instance.
- Configure the application routes.



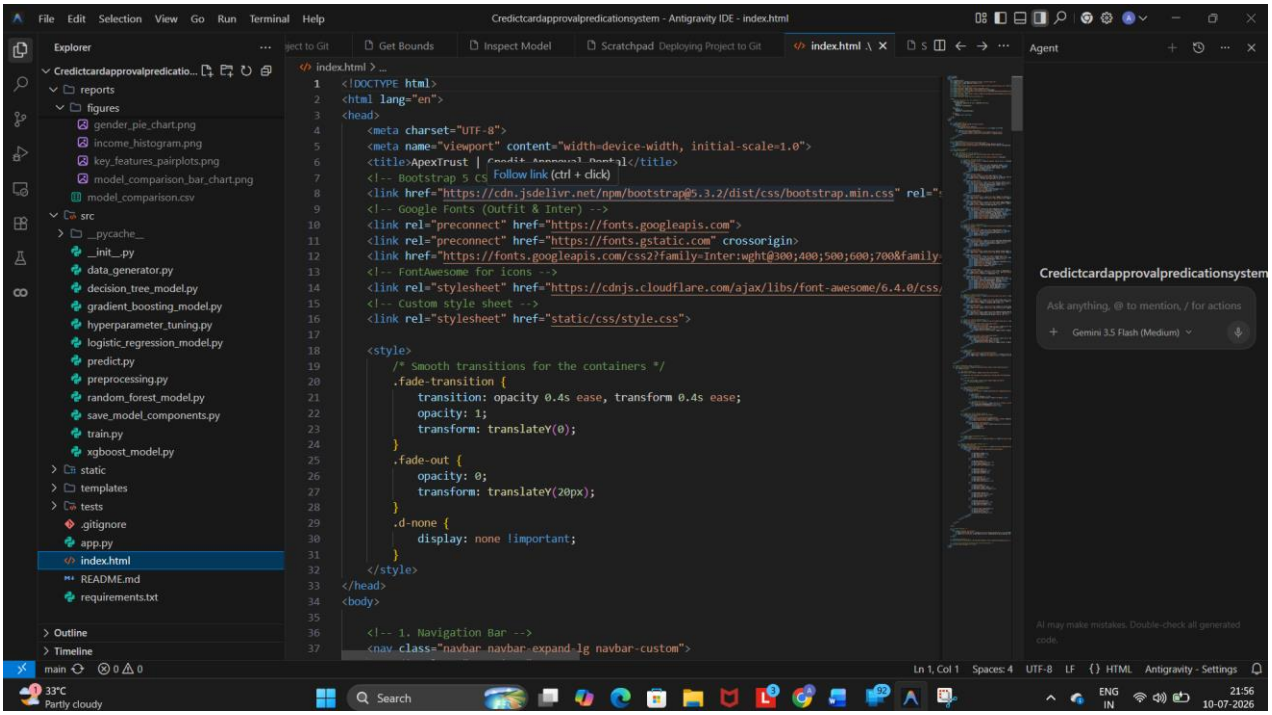
Activity 3.1: Developing the Main Application Logic in app.py

The main application logic is implemented in the **app.py** file, which serves as the core of the entire system. Initially, the required Python libraries such as Flask, Pandas, NumPy, Joblib, and Scikit-learn are imported into the application. The pre-trained machine learning model is then loaded using the Joblib library, enabling the application to perform predictions without retraining the model each time the system is executed.

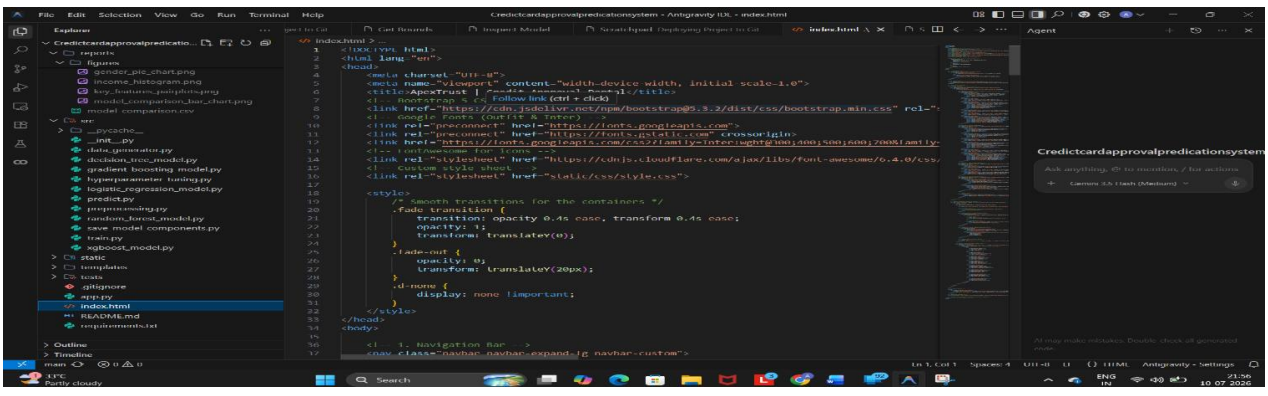
The Flask application is configured with the necessary routes to manage user interactions. The home route (/) renders the main webpage containing the credit card application form, while the prediction route (/predict) accepts the applicant's details submitted through the form. The backend validates the received input, converts it into the required numerical format, and applies the same preprocessing techniques used during model training to maintain consistency between training and prediction.

After preprocessing the input data, the trained machine learning model predicts whether the credit card application should be approved or rejected. The prediction result is then sent back to the frontend and displayed to the user in an easy-to-understand format. Proper exception handling and

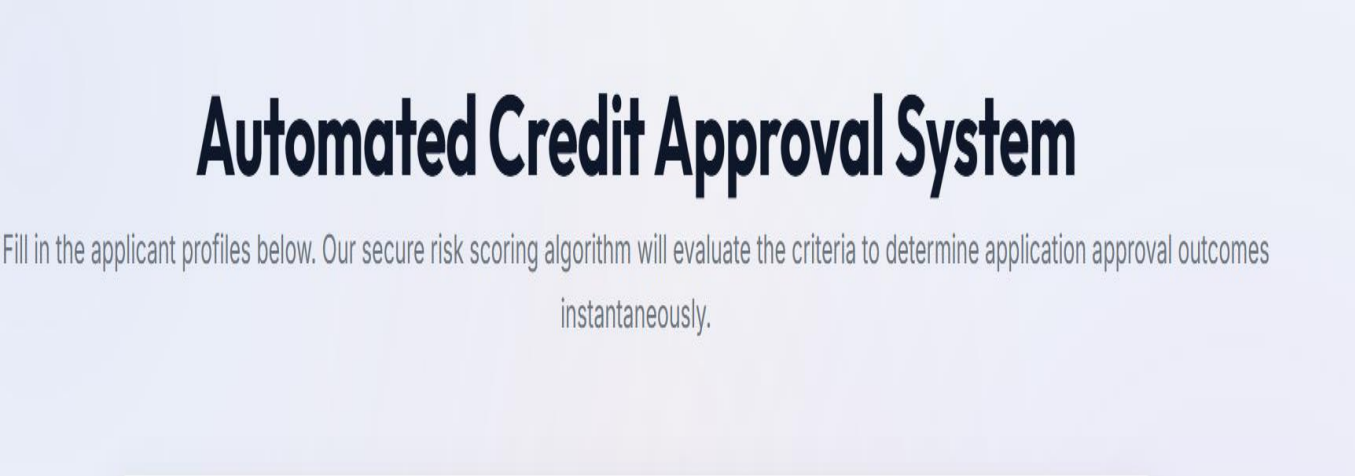
input validation mechanisms are also implemented to prevent runtime errors and ensure that invalid or incomplete user inputs are handled gracefully. Finally, the application is thoroughly tested to verify that all routes function correctly and that prediction results are generated accurately.



The user interface is developed by creating an HTML page that contains the application title, project description, and an input form for collecting applicant information. The form includes all the required fields, such as age, gender, annual income, employment status, credit history, loan amount, marital status, and other attributes required by the machine learning model. Bootstrap components are incorporated to create a clean and responsive layout, while CSS is used to enhance the appearance of the webpage with proper spacing, colors, fonts, and alignment.

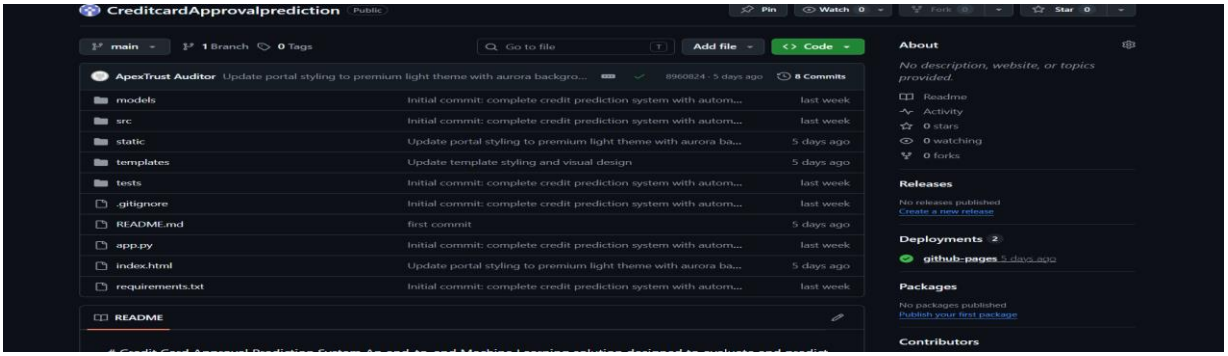


JavaScript is integrated into the frontend to improve user interaction by performing basic input validation before the form is submitted to the Flask backend. Once the user submits the application details, the data is transferred to the backend, where the trained machine learning model generates the prediction. The prediction result is then dynamically displayed on the webpage, clearly indicating whether the credit card application has been **Approved** or **Rejected**. Appropriate colors and visual indicators are used to make the prediction result easily understandable. The complete frontend is tested to ensure proper responsiveness, smooth navigation, accurate data submission, and seamless communication with the backend application



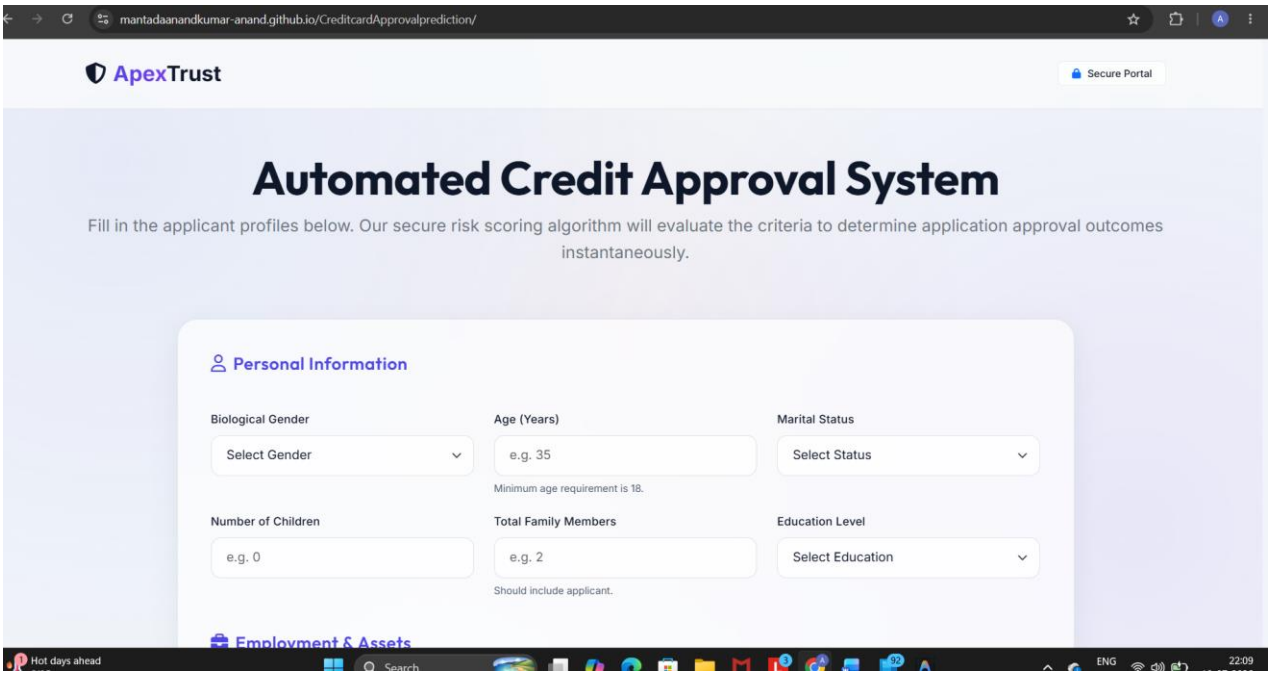
Milestone 5: Project Deployment

Milestone 5 focuses on preparing the Credit Card Approval Prediction System for deployment and publishing the complete project on GitHub. The project is organized by verifying the folder structure, ensuring all required source code, machine learning models, datasets, documentation, and dependency files are included. The application is tested locally using the Flask development server to confirm that all functionalities, including user input processing and credit card approval prediction, work correctly before publishing.



Activity 5.1: Local Testing and Verification

The application is executed locally using the Flask development server to verify its functionality. All project dependencies are installed using the requirements.txt file, and the application is started through the app.py file. The homepage, input form, prediction process, and output results are thoroughly tested to ensure the system performs accurately without errors. Any bugs identified during testing are corrected before the final project is prepared for submission.



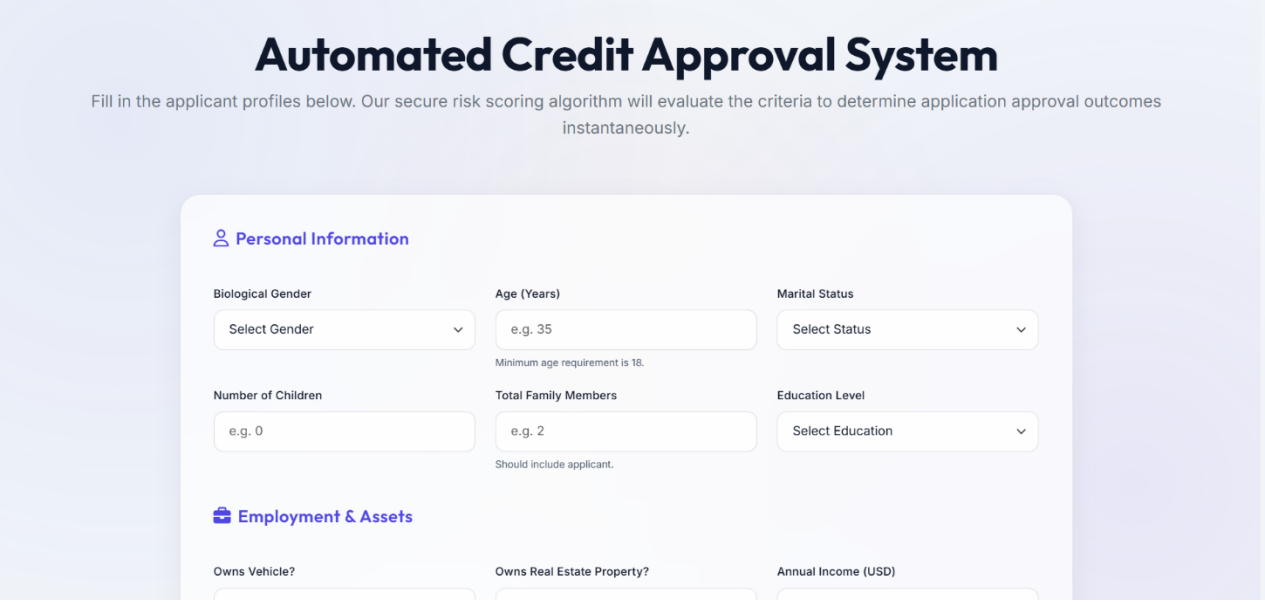
Activity 5.2: GitHub Deployment

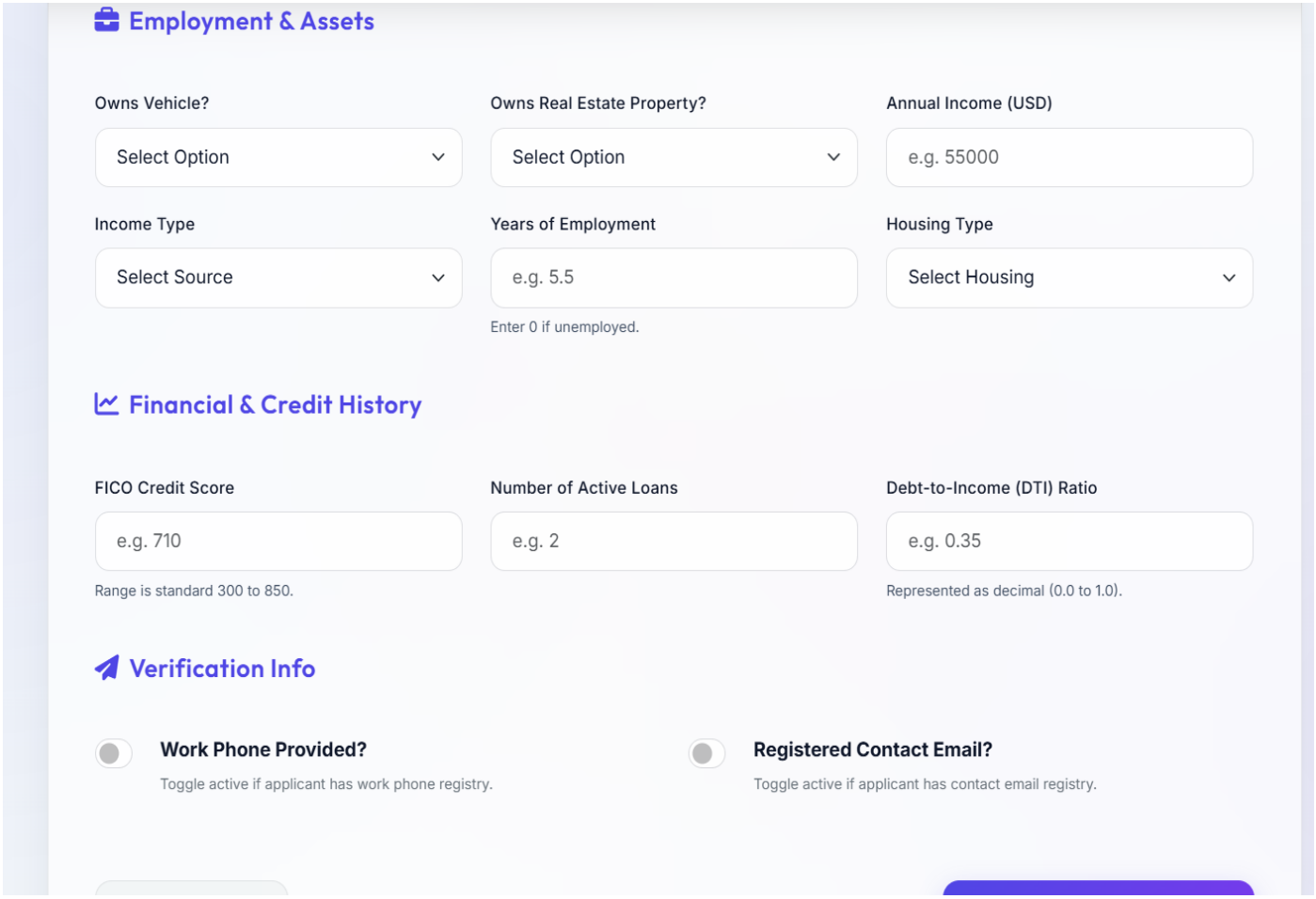
After successful testing, the entire project is uploaded to GitHub for version control and project sharing. A Git repository is initialized, and all project files, including the source code, trained machine learning model, documentation, screenshots, and dependency files, are committed to the repository. The repository is then linked to GitHub, and the project is pushed to the remote repository. A comprehensive README.md file is included to describe the project objectives, technology stack, installation procedure, execution steps, folder structure, and usage instructions. This enables reviewers and developers to access, download, and execute the project easily from the GitHub repository while maintaining proper version control and collaboration support.

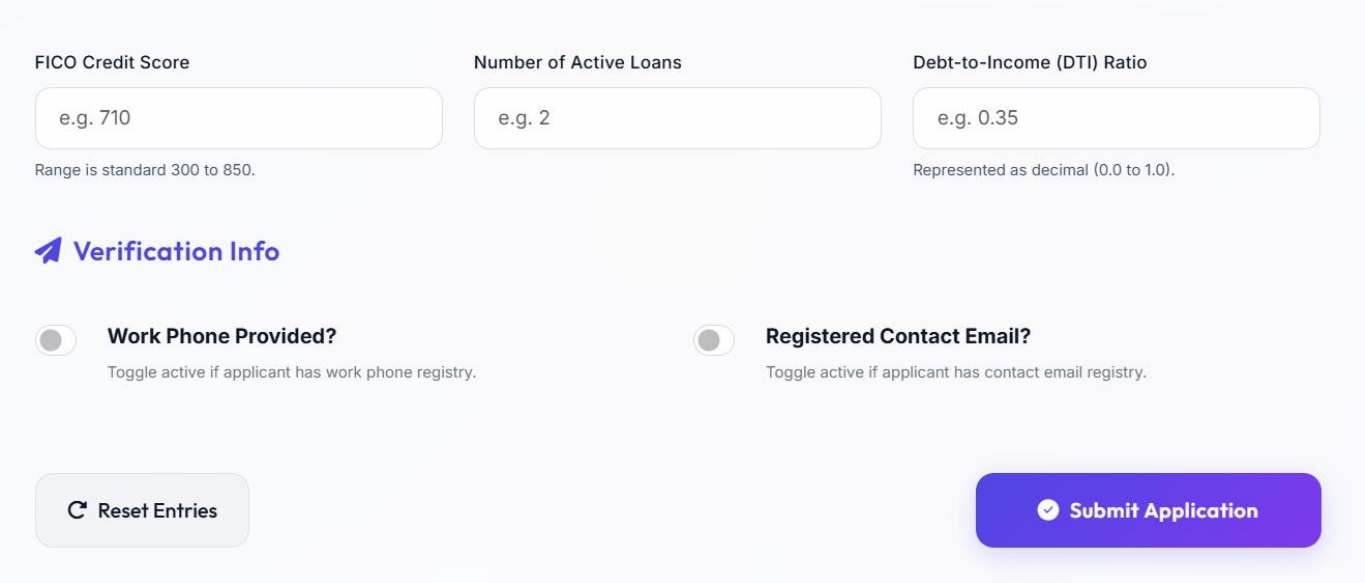
Exploring the Website's Web Pages

Home Page

The Home Page serves as the main interface of the Credit Card Approval Prediction System. It provides users with a simple and responsive interface where applicant details can be entered for prediction. The page contains the project title, input form, and navigation elements, making the application easy to understand and use. The interface is developed using HTML, CSS, and Bootstrap to ensure compatibility across different devices and screen sizes.

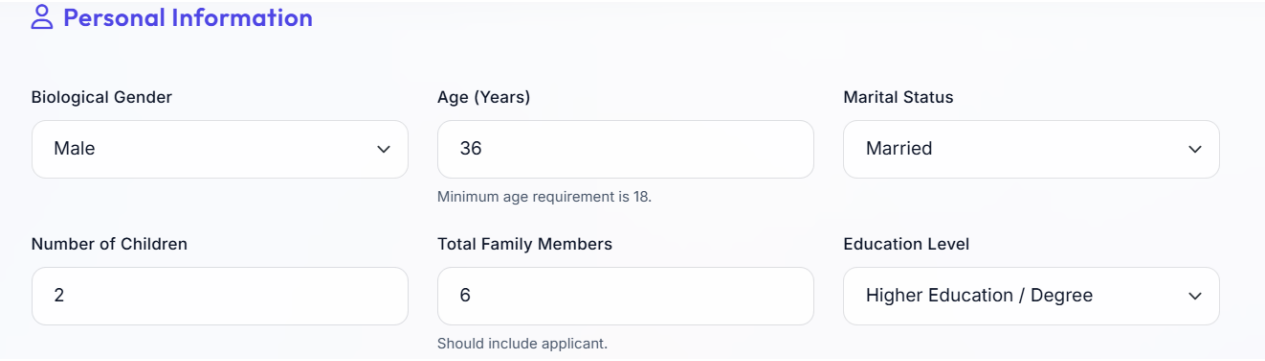


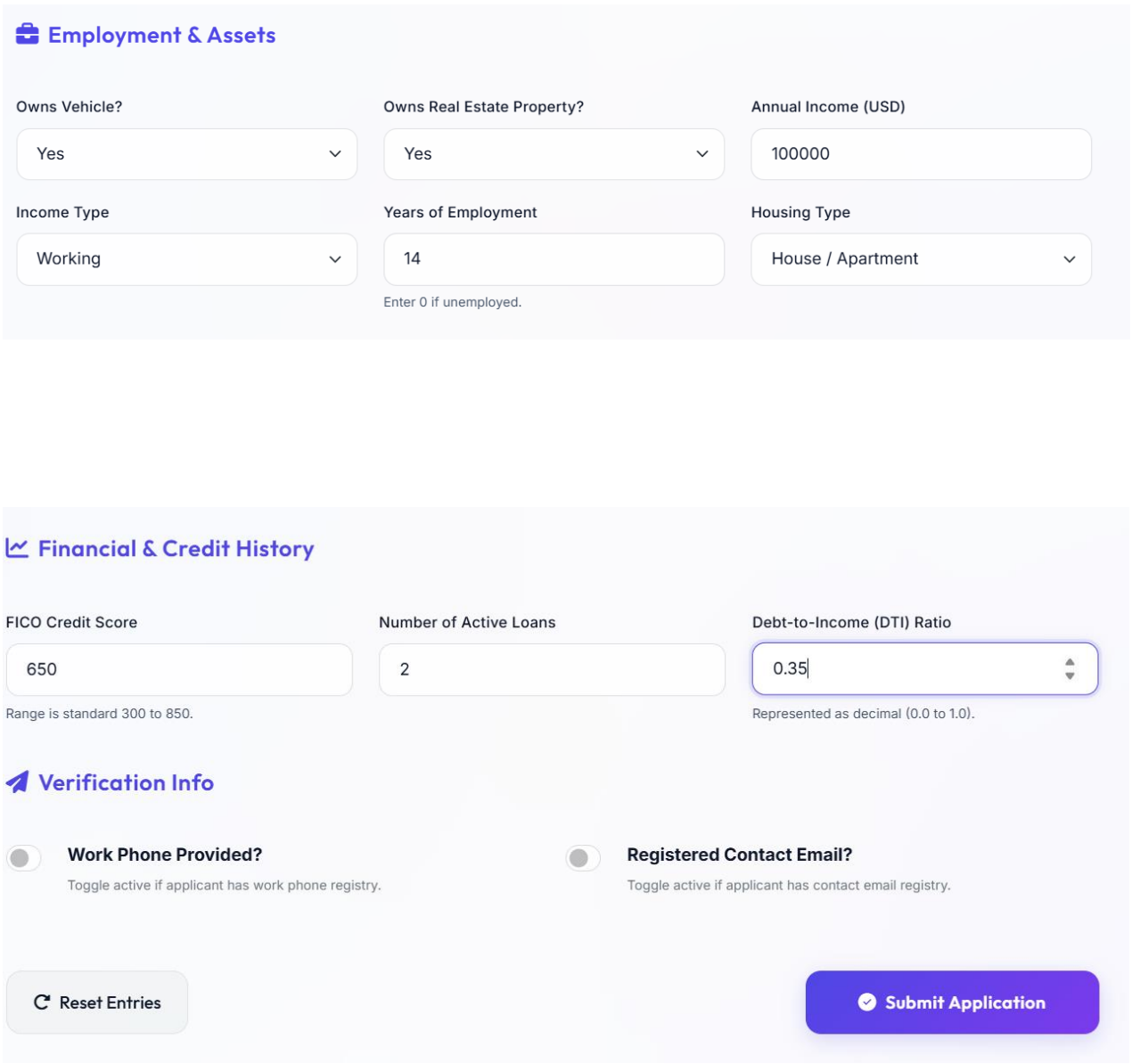

Employment & Assets
Owns Vehicle?
Select Option
Owns Real Estate Property?
Select Option
Annual Income (USD)
e.g. 55000
Income Type
Select Source
Years of Employment
e.g. 5.5
Enter 0 if unemployed.
Housing Type
Select Housing
Financial & Credit History
FICO Credit Score
e.g. 710
Range is standard 300 to 850.
Number of Active Loans
e.g. 2
Debt-to-Income (DTI) Ratio
e.g. 0.35
Represented as decimal (0.0 to 1.0).
Verification Info
Work Phone Provided?
Toggle active if applicant has work phone registry.
Registered Contact Email?
Toggle active if applicant has contact email registry.



Input Section

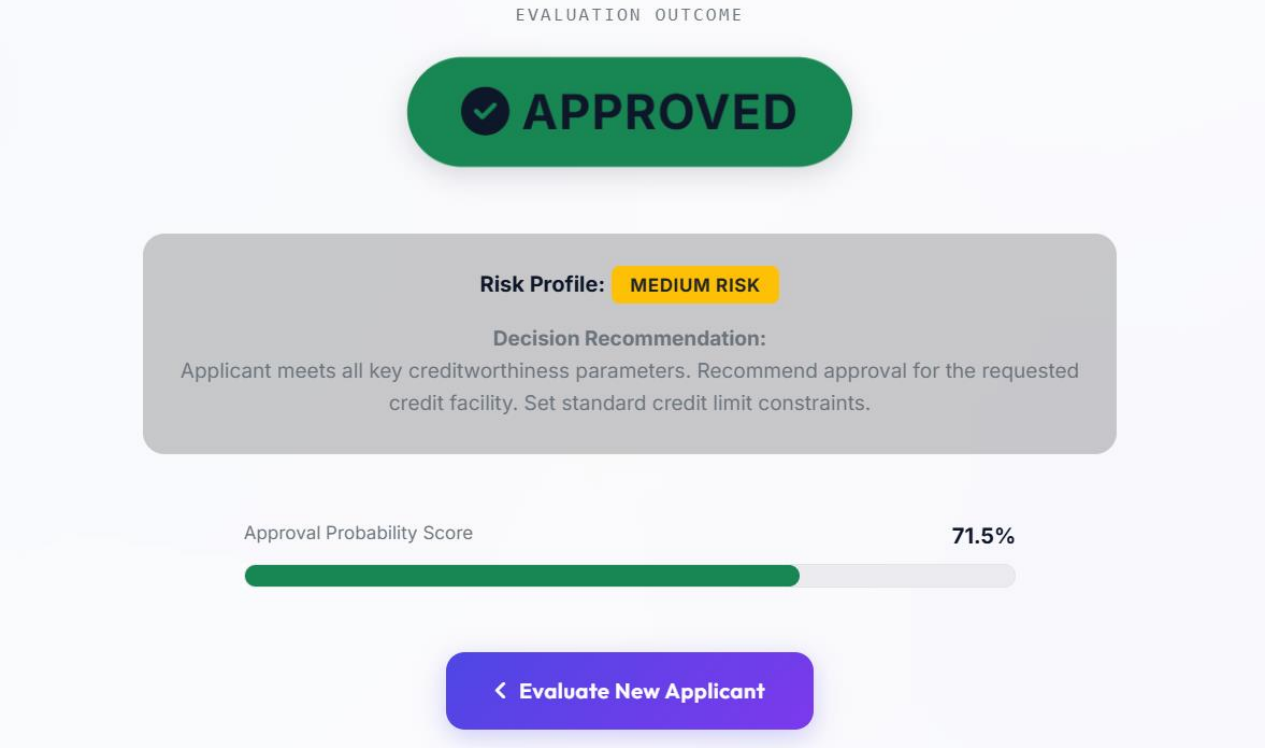
The Input Section allows users to enter the required applicant information used by the machine learning model for prediction. The form includes fields such as Age, Gender, Annual Income, Employment Status, Credit History, Loan Amount, Marital Status, and other relevant attributes. Once all the details are entered, the user clicks the **Predict** button to submit the information to the Flask backend, where the trained model processes the data and generates the prediction.





Results Section

After the prediction is completed, the Results Section displays the outcome generated by the machine learning model. The system clearly indicates whether the applicant's credit card request is **Approved** or **Rejected** based on the provided information. The prediction result is displayed in a clean and user-friendly format, allowing users to understand the decision easily. The webpage dynamically updates the result without affecting the overall user experience.



Profile Audit Log

Biological Gender	Male	Age (Years)	36
Marital Status	Married	Total Family Members	6
Education Level	Higher Education / Degree	Housing Arrangement	House / Apartment
Owns Vehicle?	Yes	Owns Real Estate?	Yes
Annual Income	$100,000.00	Employment Length (Years)	14
FICO Credit Score	650	Debt-to-Income (DTI)	35.0%
Active Loan Count	2	Work Phone Provided?	Yes
Contact Email Registered?	Yes	Income Source Type	Working

Conclusion

The **Credit Card Approval Prediction System** is a machine learning–based web application developed using Python, Flask, Scikit-learn, HTML, CSS, Bootstrap, and JavaScript. The system predicts whether a credit card application is likely to be approved by analyzing applicant information through a trained machine learning model. The application provides a simple and interactive interface for users while delivering fast and accurate predictions. The project demonstrates the practical implementation of data preprocessing, machine learning model development, web application integration, and GitHub-based project management. In the future, the system can be enhanced by incorporating larger datasets, advanced machine learning algorithms, real-time banking data integration, user authentication, and cloud deployment to further improve prediction accuracy, scalability, and usability.